



Optimal energy scheduling under time-of-use tariffs

Nicholas G. Hall¹ · Marc E. Posner²

Received: 10 March 2025 / Accepted: 7 January 2026 / Published online: 18 February 2026
© The Author(s) 2026

Abstract

The efficient use of energy can offer substantial cost savings to companies. Precise scheduling is an effective way to improve energy efficiency, since it typically requires little capital investment and has a short payback period. However, making optimal scheduling decisions can be complicated by frequent price changes in the marketplace. This creates a large-scale energy cost minimization problem, which can be difficult to solve. We study this problem for a manufacturing or computer resource with discretely variable speeds, where energy usage increases with speed. We provide a scalable optimal algorithm for minimizing energy cost with discrete speeds under frequently changing energy prices. The result is a more precise, faster and more scalable planning tool for scheduling to control energy costs than is currently available.

Keywords Manufacturing and scheduling · Computer processing · Time-of-use tariffs · Strongly polynomial time algorithm

Mathematics Subject Classification Production/scheduling—sequencing, deterministic, single machine · Natural resources—energy

1 Introduction

This work describes an efficient procedure to optimize energy-related production scheduling decisions for adjustable speed manufacturing equipment in the presence of *time-of-use* tariffs. Time-of-use tariffs apply different prices to the same energy usage, depending on the time of use. Energy suppliers use tariffs to balance demand for energy over time and achieve sustainability goals. Energy pricing markets feature frequently changing time-of-use tariffs and knowledge of future prices. The time-of-use tariffs literature is reviewed by Giglio et al. (2017) and by Catanzaro et al. (2023). An important feature of the energy marketplace is the ability to track prices using, for example, Smartgrid (U.S. Department of Commerce, 2014). Day ahead energy prices are available for various markets, for example: New England at <https://www.iso-ne.com/isoexpress/web/reports/pricing>; Texas at <https://www.ercot.com/mktinfo/dam>; and Spain and Por-

tugal at <https://www.omie.es/en/market-results/daily/daily-market/daily-hourly-price>. Accurate price forecasting is also possible, based on repeating cyclical patterns in historical data (Lei & Feng, 2012).

When energy prices change over short intervals, frequent manufacturing decisions are required about task scheduling and about machine speeds. The need to make such frequent decisions generates large decision problems with corresponding computational challenges. As a benchmark with respect to solving such problems, the existing literature studies problem instances with 60 tasks (Shrouf et al., 2014) and two machine speeds and provides a metaheuristic solution procedure.

Efficient energy usage is important in both the public and private sectors. In 2022, the industrial share of national energy usage was 33% in the USA (U.S. Energy Information Administration, 2025) and 25.1% in Europe (Eurostat, 2024). Moreover, energy can be a significant component of the manufacturing cost for a company. Examples include 25–30% in iron and steel manufacturing, 32.5% in the ceramics industry, and 9–30% in pulp and paper manufacturing (Mickovic & Wouters, 2020). Energy efficiency generates life-cycle dollar savings while also reducing emissions, and it is more cost-effective to save energy through efficiency measures than to generate it (Miller, 2021). Industry standard ISO 50001 (ISO,

✉ Nicholas G. Hall
hall.33@osu.edu

¹ Fisher College of Business, The Ohio State University, 2100 Neil Avenue, Columbus, OH 43210, USA

² Department of Integrated Systems Engineering, The Ohio State University, 1971 Neil Avenue, Columbus, OH 43210, USA

2013) encourages companies to develop plans for continuous improvement of energy efficiency. Energy costs typically impose a substantial strain on the budget of data and computing centers (Albers, 2010).

Variable speed manufacturing equipment can provide substantial energy cost savings (Carlucci et al., 2023). The authors demonstrate that the use of variable speeds enables finding feasible solutions that satisfy a fixed limit on energy usage; however, they do not consider time-of-use tariffs. The modeling of energy usage with *discrete speeds* is motivated by various practical considerations. In practice, only a finite set of discrete speeds is available (Albers, 2010). Commercial examples of discrete speed computer processors include Intel SpeedSelect, Motorola MPC5200, and Transmeta Crusoe. Mochocki et al. (2002) observe that many variable voltage processors provide only a limited number of voltage levels. They also provide an example where the difference in energy usage between continuous and discrete voltage levels is large. Hence, the use of a continuous approximation for the available voltage levels results in poor energy efficiency and cost control. Liu (2024) provides an updated survey of the use of dynamic voltage scaling for power management.

Models for scheduling problems with discrete speeds arise from two environments. In the first environment, the machine speed is determined by a discrete resource, as discussed by Chen et al. (1997), Grigoriev et al. (2007), Grigoriev and Uetz (2009), and Hsieh et al. (2015). In the second environment, the machine has a discrete set of speed processing modes, as discussed by Daniels and Mazzola (1994), Daniels et al. (1996, 1997) Jansen et al. (2001), Chen (2004), Rao and Vrudhula (2005), Li and Yao (2006), Cao et al. (2006), Fang et al. (2011), and Zhang and Zhang (2013). However, none of these works consider variable energy pricing under time-of-use tariffs.

Related work where some jobs are not initially available for processing includes Kwon and Kim (2005), Zhang et al. (2006), Mouzon et al. (2007), and Mouzon and Yildirim (2008). In the last two papers, there is only one processing speed. None of these works consider time-of-use tariffs. Antoniadis et al. (2017) consider release dates, deadlines, time-of-use tariffs, and a dynamic speed limit but use a continuous-speed model.

Only the following papers consider both discrete speeds and time-of-use tariffs. For these conditions, Moon et al. (2013) consider timing and machine assignment issues. Zhang et al. (2014) consider the timing of on- and off-modes. A general mixed integer formulation that allows for uncertain demand and various production modes is proposed by Mitra et al. (2014). Sharma et al. (2015) consider a job shop that produces multiple parts on multiple machines. The model is solved approximately using a metaheuristic. All of these papers assume that tasks must be completed by a planning horizon.

The research that is closest to the current work is Shrouf et al. (2014). They consider an energy scheduling problem with time-of-use tariffs, nonpreemptive tasks, and a planning horizon where the objective is to minimize total energy cost. However, they allow only four possible machine states: processing, idle, and in transition between those states in either direction. They do not consider variable speeds which, based on industry practice, motivate our work. They impose some additional restrictions on the transition process. Their solution process is heuristic, using a genetic algorithm. For problem instances with up to 60 tasks and 135 time periods, they demonstrate that their genetic algorithm delivers close to optimal results. By contrast, the model we consider allows up to 200 tasks, 500 periods, and 6 machine speeds and is solved optimally using a *strongly polynomial time* procedure (as defined by Grötschel et al., 1988). As a consequence, our work provides a more precise, faster, and more generally applicable planning tool for scheduling to control energy costs than is currently available.

This paper is organized as follows. In Sect. 2, we provide notation and assumptions, and present some preliminary results. The theory that supports our algorithm is developed in Sect. 3. Section 4 provides optimal approaches for various one-task problems. Section 5 generalizes the results of the prior sections to consider multiple tasks that arrive throughout the planning horizon. The efficiency of our algorithm is tested against the commercial IP solver CPLEX in Sect. 6. Section 7 describes two generalizations of our problem that are formally intractable. Section 8 provides a conclusion and several directions for future research. Several proofs appear in an Online Appendix.

2 Preliminaries

We consider a set of tasks $N = \{1, \dots, n\}$ that must be processed nonpreemptively on a single machine. For task $h \in N$, let p_h denote its nominal processing time, i.e. the time required for processing the task at speed 1. Also, let $P = \sum_{h=1}^n p_h$. Task $h \in N$ becomes available for processing at its release date r_h . The tasks are indexed such that $0 \leq r_1 \leq \dots \leq r_n$, with ties broken arbitrarily.

The planning horizon has τ time intervals $[t_{i-1}, t_i)$ for $i \in T = \{1, \dots, \tau\}$, where $t_0 = 0$. The interval $[t_{i-1}, t_i)$ is referred to as interval i , and has an energy cost of c_i per kWh for work performed in the interval. A feasible solution must complete all jobs by time t_τ . Also, we define the interval width $\Delta_i = t_i - t_{i-1}$, for $i \in T$.

The machine has q possible speeds, $s_1 > \dots > s_q = 0$. For a given speed s_j , $j \in Q = \{1, \dots, q\}$, the energy usage is e_j per unit time. If a task h with nominal processing time p_h is processed entirely at speed s_k , then its actual processing time is p_h/s_k , and the total energy usage for processing it is

$e_k p_h / s_k$. The energy cost per unit time at time $t \in [t_{i-1}, t_i)$ at speed s_j is $c_i e_j$. Further, if all this processing occurs within interval i , the total energy cost for processing the task is $c_i e_j p_h / s_j$. If $e_j \leq e_{j+1}$, for $1 \leq j \leq q-1$, then it is not optimal to process at speed s_{j+1} . Consequently, without loss of generality we assume that $e_1 > \dots > e_q > 0$.

Let $\bar{Q} = \{1, \dots, q-1\}$ be the set of *active* speeds, index q represent the condition that the machine is sleeping, and index $q+1$ represent the condition that the machine is powered down. *Idle time* occurs when the machine is sleeping or powered down. The machine speeds $s_1, \dots, s_q$ can be changed at any time. There may be times when no tasks are available, or tasks are available but it is not economical to process them. For such times, the scheduler may want to put the machine into sleep mode with an energy usage rate of $e_q > 0$ and a speed of $s_q = 0$.

When there are no available tasks, there is the further option to power down the machine. For this situation, $s_{q+1} = e_{q+1} = 0$, where we let $\bar{Q}' = \bar{Q} \cup \{q+1\}$. While the machine is powered down, no energy is used. A cost $K \geq 0$ is incurred to power up and restart the machine. This approach follows Zhang and Chiong (2016) in assuming that a machine cannot be powered down until all currently available tasks are completed. They motivate this assumption by the fact that frequent on and off switching causes damage to machine tools. Orientech Automation Solutions (2020) identifies specific problems that arise from powering down a machine.

The objective is to minimize the cost of energy used plus the cost of machine power-ups, subject to the constraint that all jobs are completed by a common deadline t_τ . Without loss of generality, we assume that initially the machine is off.

We now develop a preprocessing procedure to simplify the problem formulation. The first result shows that an optimal task sequence can be found easily.

Theorem 1 *There exists an optimal solution where the tasks are scheduled in earliest release date order.*

Proof Suppose that in schedule σ task i is processed directly after task j , but $r_i < r_j$, where $i, j \in N$. Consider a new schedule where i and j are processed in the same time periods, but i is processed first. Since they are processed in the same periods, the cost and total completion time are unchanged. Thus, we can consider a new schedule where i is processed before j . This argument can be repeated for each instance where the jobs of σ are not processed in earliest release date order. $\square$

In view of Theorem 1, the difficulty of solving the problem arises mainly from determining the machine speeds across time periods with different energy costs. Since the processing order is specified by Theorem 1, tasks that must be processed without the machine powering down can be regarded as one task. This suggests the following procedure to reduce the number of tasks.

Procedure TaskReduce:

0. Input the number of tasks n , p_h and r_h for $h = 1, \dots, n$, and s_1 . Set $h = 1$.
1. If $r_h + p_h/s_1 \geq r_{h+1}$, then set $p_h = p_h + p_{h+1}$, and $p_l = p_{l+1}$, $r_l = r_{l+1}$ for $l = h+1, \dots, n-1$, and $h = h+1$, and $n = n-1$.
2. If $h < n-1$, then set $h = h+1$ and go to Step 1. Otherwise, stop.

Procedure TaskReduce combines tasks such that all the remaining tasks have the following property: *If a task is processed as soon as possible and at the fastest speed, then there is idle time before the next task is released.*

For the new task set, create a new interval for each $r_h \notin \{t_0, \dots, t_\tau\}$. If $t_i < r_h < t_{i+1}$, then set $\tau = \tau + 1$, and $t_l = t_{l-1}$, $c_l = c_{l-1}$, for $l = i+2, \dots, \tau$, and $t_{i+1} = r_h$. Consequently, the earliest start time of each task is at the beginning of some interval. To record a task's earliest start interval, let $\rho_h \in T$ such that $t_{\rho_h-1} = r_h$, for $h \in N$.

Remark 1 There exists an optimal solution where, in any interval that contains idle time, the idle time occurs at consecutive time units.

Shifting task processing within an interval does not affect the energy cost. Hence, there exists an optimal solution where a power-up occurs as late as possible in an interval and a power-down occurs as early as possible. If an interval has both a power-up and power-down, for formulation purposes only, without loss of generality either the power-up occurs at the end of the prior interval or the power-down occurs at the start of the next interval. As a result, an interval has at most one power-up or power-down.

To formulate this problem, let

x_{ij} = the time spent working at speed $j \in \bar{Q}$ in interval $i \in T$

$w_i = \begin{cases} 1, & \text{if a power-down is performed during interval } i \in T \\ 0, & \text{otherwise} \end{cases}$

$y_i = \begin{cases} 1, & \text{if a power-up is performed during interval } i \in T \\ 0, & \text{otherwise.} \end{cases}$

A linear integer programming formulation is

$$(\mathcal{P}) \text{ minimize } z(x, y, w) = \sum_{i \in T} c_i \sum_{j \in \bar{Q}} e_j x_{ij} + K \sum_{i \in T} y_i$$

$$\text{s.t. } \sum_{\rho_h \leq i \leq \tau} \sum_{j \in \bar{Q}} s_j x_{ij} \geq \sum_{k=h}^n p_k, \quad h \in N \quad (1)$$

$$1 \geq \sum_{i=1}^l (y_i - w_i) \geq 0, \quad l \in T \quad (2)$$

$$y_l + w_l \leq 1, \quad l \in T \setminus \{1\} \quad (3)$$

$$\Delta_l \left[\sum_{i=1}^{l-1} (y_i - w_i) + y_l \right] - \sum_{j \in Q} x_{lj} \geq 0, \quad l \in T \quad (4)$$

$$\sum_{j \in Q} x_{lj} - \Delta_l \left[\sum_{i=1}^{l-1} (y_i - w_i) - w_l \right] \geq 0, \quad l \in T \quad (5)$$

$$\sum_{j \in Q} s_j x_{1j} - w_1 p_1 \geq 0, \quad (6)$$

$$\begin{aligned} \sum_{i=1}^l \sum_{j \in Q} s_j x_{ij} - \left[w_l - 1 + \sum_{i=1}^{l-1} (y_i - w_i) \right] \times \\ \sum_{h \in \{k \in N \mid \rho_k \leq l\}} p_h \geq 0, \quad l \in T \setminus \{1\} \quad (7) \\ x_{ij} \geq 0, \quad i \in T, j \in Q \\ w_i, y_i \in \{0, 1\}, \quad i \in T. \end{aligned}$$

In problem $(\mathcal{P})$, the objective minimizes the total cost of energy plus power-ups. Constraint (1) prevents a task from being started before its release date. Constraint (2) requires that the difference at any time between the number of power-ups and power-downs is at most one. Constraint (3) ensures that at most one power-up or power-down occurs in any interval, which enforces (4) and (5). Constraint (4) prevents processing if the machine is powered-down for the entire interval. Constraint (5) ensures processing throughout an interval if the machine is powered up for the entire interval. Constraints (6) and (7) require that the machine not be powered down when work is available. As discussed in the Introduction, our objective function is similar to that of Shrouf et al. (2014). However, our use of a set of discrete processing speeds, Q , represents a significant generalization of their model. We also allow both a sleep state and an off (powered-down) state instead of a single “idle” state. In our model, transitions from one state to another are instantaneous. Hence, we do not model transition states.

3 Optimality conditions for the one-task problem

In this section, we develop intuition about the structure of problem $\mathcal{P}$ by considering an instance with only one task. This task is released at time zero, and the machine is initially powered up. Let

x_{ij} = the time spent working at speed $j \in Q$
in interval $i \in T$

$w_i = \begin{cases} 1, & \text{if there is a power-down in interval } i \in T \\ 0, & \text{otherwise.} \end{cases}$

A linear integer programming formulation of this problem is

$$\begin{aligned} (\mathcal{P}_1) \quad & \text{minimize } z(x, w) = \sum_{i \in T} c_i \sum_{j \in Q} e_j x_{ij} \\ & \text{s.t. } \sum_{i \in T} \sum_{j \in Q} s_j x_{ij} = p_1 \end{aligned} \quad (8)$$

$$\begin{aligned} \Delta_{i+1} \left(1 - \sum_{l=1}^i w_l \right) &\geq \sum_{j \in Q} x_{i+1j} \geq \Delta_{i+1} \left(1 - \sum_{l=1}^{i+1} w_l \right), \\ i &= 1, \dots, \tau - 1 \\ x_{ij} &\geq 0, \quad i \in T, j \in Q \\ w_i &\in \{0, 1\}, \quad i \in T. \end{aligned} \quad (9)$$

Since the power-down occurs only after the task is completed, the variable y is not needed. Equation (8) replaces (1) because there is only one task. Also, (9) replaces (2)–(7). We let x^* denote an optimal solution, since it uniquely defines $w_i, i \in T$.

Even though it is a restricted version of Problem $\mathcal{P}$, Problem $\mathcal{P}_1$ cannot be written as a convex program. This is illustrated by the following example, where a locally minimum solution is not a global minimum.

Example 1 $\tau = 3, t = (10, 15, 20), c = (2, 9, 1), q = 2, s = (3, 1), e = (4, 1)$, and $P = 30$.

Consider the solution with $x_{12} = 10, x_{22} = x_{31} = 5$, and $x_{ij} = 0$ otherwise. Since this solution completes the required 30 units of work at $t_\tau = 20$, the solution is feasible. The energy cost is $2 \cdot 1 \cdot 10 + 9 \cdot 1 \cdot 5 + 1 \cdot 4 \cdot 5 = 85$. This solution is locally optimal since reducing the speed in interval 3 prevents the solution from finishing by the planning horizon of 20, and increasing the speed in intervals 1 or 2 increases the energy cost.

However, the globally optimal solution is $x_{11} = 10$ and $x_{ij} = 0$ otherwise, where processing completes at time 10. The associated energy cost is $2 \cdot 4 \cdot 10 = 80$.

Notwithstanding Example 1, Problem $\mathcal{P}_1$ can be solved by a series of linear programs, each having a different specified completion interval $\omega \in T$. Given that processing completes in interval ω , a linear programming formulation is:

$$(\mathcal{P}_1(\omega)) \quad \text{minimize } z_\omega(x) = \sum_{i=1}^{\omega} c_i \sum_{j \in Q} e_j x_{ij} \quad (10)$$

$$\text{s.t. } \sum_{i=1}^{\omega} \sum_{j \in Q} s_j x_{ij} = p_1 \quad (11)$$

$$\sum_{j \in Q} x_{ij} = \Delta_i, \quad i = 1, \dots, \omega - 1 \quad (12)$$

$$\sum_{j \in Q} x_{\omega j} \leq \Delta_\omega \quad (13)$$

$$x_{ij} \geq 0, \quad i = 1, \dots, \omega, j \in Q. \quad (14)$$

Thus, Problem $\mathcal{P}_1$ can be solved by finding solutions to τ linear programs, $\mathcal{P}_1(1), \dots, \mathcal{P}_1(\tau)$. Let $W = \{1, \dots, \omega\}$, for

a given $\omega \in \{1, \dots, T\}$. We develop a more efficient method that, unlike known methods for general linear programming, runs in strongly polynomial time. To support our approach, some preliminary results are needed. The first result shows that the set of potentially optimal (speed, energy usage) pairs describes a piecewise convex function.

Theorem 2 *There exists an optimal solution x^* to Problem $\mathcal{P}_1$, where $x_{iu}^* > 0$ for some $i \in W$ and $1 \leq u \leq q$ implies that*

$$\frac{e_u - e_{u_2}}{s_u - s_{u_2}} < \frac{e_{u_1} - e_u}{s_{u_1} - s_u}, \text{ for } 1 \leq u_1 < u < u_2 \leq q. \quad (15)$$

Proof We establish the inequality by showing that, if it does not hold, then there exists an optimal solution to problem $\mathcal{P}_1$ where $x_{iu} = 0$, for $i \in T$. Hence, speed u can be removed from consideration.

Suppose for some $k \in T$ and $u_1, u, u_2 \in Q$, we have $u_1 < u < u_2$, $(e_u - e_{u_2})/(s_u - s_{u_2}) \geq (e_{u_1} - e_u)/(s_{u_1} - s_u)$, and $x_{ku}^* > 0$. Consider the alternative solution

$$x_{ij}^0 = \begin{cases} x_{ku_1}^* + (s_u - s_{u_2})x_{ku}^*/(s_{u_1} - s_{u_2}), & i = k, j = u_1 \\ 0, & i = k, j = u \\ x_{ku_2}^* + (s_{u_1} - s_u)x_{ku}^*/(s_{u_1} - s_{u_2}), & i = k, j = u_2 \\ x_{ij}^*, & \text{otherwise.} \end{cases}$$

Because

$$\begin{aligned} x_{ku_1}^0 + x_{ku}^0 + x_{ku_2}^0 &= x_{ku_1}^* + \frac{s_u - s_{u_2}}{s_{u_1} - s_{u_2}}x_{ku}^* + x_{ku_2}^* + \frac{s_{u_1} - s_u}{s_{u_1} - s_{u_2}}x_{ku}^* \\ &= x_{ku_1}^* + x_{ku}^* + x_{ku_2}^*, \end{aligned}$$

x^0 completes at the same time as x^* . Further, since

$$s_{u_1} \frac{s_u - s_{u_2}}{s_{u_1} - s_{u_2}}x_{ku}^* + s_{u_2} \frac{s_{u_1} - s_u}{s_{u_1} - s_{u_2}}x_{ku}^* = s_u x_{ku}^*,$$

x^0 completes the same work as x^* . Finally, because $(e_u - e_{u_2})/(s_u - s_{u_2}) \geq (e_{u_1} - e_u)/(s_{u_1} - s_u)$ implies

$$e_{u_1} \frac{s_u - s_{u_2}}{s_{u_1} - s_{u_2}}x_{ku}^* + e_{u_2} \frac{s_{u_1} - s_u}{s_{u_1} - s_{u_2}}x_{ku}^* \leq e_u x_{ku}^*,$$

the energy cost of x^0 is no larger than that of x^* . Consequently, x^0 dominates x^* , and $x_{iu} = 0$ for all $i \in T$. $\square$

From Theorem 2, we assume that (15) holds for all available (speed, energy usage) pairs.

We now develop some properties of an optimal solution. The next result establishes that there is an optimal solution where the machine processes at multiple speeds in at most one interval. Moreover, convexity ensures that these speeds are adjacent in the ordered list of speeds.

Lemma 1 *There exists an optimal solution x^* to Problem $\mathcal{P}_1$ where $\left| \{(i, j) \mid x_{ij}^* > 0, i \in W, j \in \bar{Q}\} \right| \in \{1, 2\}$ and equals 2 for at most one $i \in W$. Further, if $x_{ik_1}^* > 0$ and $x_{ik_2}^* > 0$ for $k_1, k_2 \in \bar{Q}$, then $|k_1 - k_2| = 1$.*

Proof Theorem 2 shows that energy usage is an increasing convex function of speed. This implies that there exists an optimal solution x^* where if $x_{ik_1}^* > 0$ and $x_{ik_2}^* > 0$ for $k_1, k_2 \in \bar{Q}$, then $|k_1 - k_2| = 1$.

Suppose that for some optimal solution x^* , we have $x_{k_1 l}^* > 0$, $x_{k_1 l+1}^* > 0$, $x_{k_2 u}^* > 0$, and $x_{k_2 u+1}^* > 0$, where $k_1, k_2 \in W$, and $l, u \in \bar{Q}$.

Define k_1 and k_2 such that $c_{k_1}(e_l - e_{l+1})/(s_l - s_{l+1}) \geq c_{k_2}(e_u - e_{u+1})/(s_u - s_{u+1})$. Consider the alternative solution

$$x_{ij}^0 = \begin{cases} x_{k_1 l}^* - \epsilon, & i = k_1, j = l \\ x_{k_1 l+1}^* + \epsilon, & i = k_1, j = l+1 \\ x_{k_2 u}^* + \epsilon(s_l - s_{l+1})/(s_u - s_{u+1}), & i = k_2, j = u \\ x_{k_2 u+1}^* - \epsilon(s_l - s_{l+1})/(s_u - s_{u+1}), & i = k_2, j = u+1 \\ x_{ij}^*, & \text{otherwise,} \end{cases}$$

where $\epsilon = \min\{x_{k_1 l}^*, x_{k_2 u+1}^*(s_u - s_{u+1})/(s_l - s_{l+1})\}$.

In x^0 , the more expensive interval k_1 contains $\epsilon(s_l - s_{l+1})$ less work, whereas the less expensive interval k_2 contains $[\epsilon(s_l - s_{l+1})/(s_u - s_{u+1})](s_u - s_{u+1}) = \epsilon(s_l - s_{l+1})$ additional work. Consequently, x^0 processes all tasks and completes at the same time as x^* . Further, the energy cost saving in interval k_1 is $c_{k_1}\epsilon(e_l - e_{l+1})$, whereas the increased cost in interval k_2 is $c_{k_2}\epsilon(e_u - e_{u+1})[(s_l - s_{l+1})/(s_u - s_{u+1})]$. Because $c_{k_1}(e_l - e_{l+1})/(s_l - s_{l+1}) \geq c_{k_2}(e_u - e_{u+1})/(s_u - s_{u+1})$, x^0 incurs no more energy cost than x^* . Hence, x^0 is optimal.

Since either $x_{k_1 l}^0$ or $x_{k_2 u+1}^0$ equals zero, the number of intervals where more than one processing speed is used is at least one less for x^0 than for x^* . The above transformation can be repeated until at most one interval has two adjacent processing speeds. $\square$

Let $\bar{q} = \operatorname{argmin}_{j \in Q} \{e_j/s_j\}$ denote the speed index with the smallest ratio of energy usage to processing speed, with ties broken in favor of the faster speed. The next result shows that at most one interval is partially active, and if such an interval exists, its unique processing speed is $\bar{q}$.

Lemma 2 *There exists an optimal solution x^* to Problem $\mathcal{P}_1$ where if $0 < \sum_{j \in \bar{Q}} x_{kj}^* < \Delta_k$ for $k \in W$, then $\left| \{(i, j) \mid x_{ij}^0 > 0, j \in Q\} \right| = 1$ for $i \in W \setminus \{k\}$. Further, $x_{i\bar{q}}^* = \sum_{j \in \bar{Q}} x_{ij}^*$.*

Proof Similar to the proof of Lemma 1. $\square$

Given x^0 that satisfies Lemmas 1 and 2, let the fastest processing speed in interval $i \in W$ be

$$\gamma_i(x^0) = \min \{j \in Q \mid x_{ij}^0 > 0\}.$$

Let

$$\alpha(x^0) = \operatorname{argmax}_{i \in T} \{c_i(e_{\gamma_i(x^0)} - e_{\gamma_i+1(x^0)}) / (s_{\gamma_i(x^0)} - s_{\gamma_i+1(x^0)}) \mid \gamma_i(x^0) < q\} \quad (16)$$

be the interval that gives the greatest marginal cost savings achieved by using the next slower speed, and $\beta(x^0)$ be the corresponding current speed index for interval $\alpha(x^0)$. When clear from context, we omit (x^0) from γ , α , and β . In case of ties, without loss of generality, let α denote the earliest such interval. Also, let $\hat{e} = e_\beta - e_{\beta+1}$ and $\hat{s} = s_\beta - s_{\beta+1}$.

The next result shows that there exists an optimal solution where nonzero processing time at two adjacent speeds in the same interval only occurs in interval α .

Lemma 3 *There exists an optimal solution x^* to problem $\mathcal{P}_1$, where if $x_{ij}^* > 0$ and $x_{ij+1}^* > 0$, for some $i \in W$ and $j \in \bar{Q}$, then $i = \alpha$.*

Proof From Lemma 1, there is a second processing speed in at most one interval. Suppose that in optimal solution x^* , we have $x_{kl}^* > 0$, $x_{kl+1}^* > 0$, and $x_{\alpha\beta}^* > 0$, for some $k \in T$, $k \neq \alpha$, and $l \in \bar{Q}$. Consider the alternative solution

$$x_{ij}^0 = \begin{cases} x_{kl}^* + \epsilon, & i = k, j = l \\ x_{kl+1}^* - \epsilon, & i = k, j = l + 1 \\ x_{\alpha\beta}^* - \epsilon(s_l - s_{l+1})/\hat{s}, & i = \alpha, j = \beta \\ x_{\alpha\beta+1}^* + \epsilon(s_l - s_{l+1})/\hat{s}, & i = \alpha, j = \beta + 1 \\ x_{ij}^*, & \text{otherwise,} \end{cases}$$

where $\epsilon = \min\{x_{kl+1}^*, x_{\alpha\beta}^*\hat{s}/(s_l - s_{l+1})\}$.

In solution x^0 ,

$$\epsilon s_l - \epsilon s_{l+1} - \epsilon \frac{s_l - s_{l+1}}{\hat{s}} s_\beta + \epsilon \frac{s_l - s_{l+1}}{\hat{s}} s_{\beta+1} = 0.$$

Further, the energy cost of solution x^0 is no larger than that of x^* since

$$c_k e_l - c_k e_{l+1} - c_\alpha e_\beta \frac{s_l - s_{l+1}}{\hat{s}} + c_\alpha e_{\beta+1} \frac{s_l - s_{l+1}}{\hat{s}} \leq 0,$$

where the inequality follows from the definition of α , which requires that $c_k(e_l - e_{l+1})/(s_l - s_{l+1}) \leq c_\alpha \hat{e}/\hat{s}$. Thus, x^0 dominates x^* .

In x^0 , either $x_{kl+1}^0 = 0$, or $x_{\alpha\beta}^0 = 0$. If $x_{kl+1}^0 = 0$, then processing in interval k is entirely at speed s_l , which completes the proof. Alternatively, if $x_{\alpha\beta}^0 = 0$, then the solution x^0 has a different (α, β) pair than x^* . In the latter case, the transformation is repeated until either $x_{kl+1}^0 = 0$ or $k = \alpha$. $\square$

Given an (α, β) pair defined by (16), we now determine the processing speed in any interval. The proof is in two parts. Lemma 4 considers intervals without idle time. Lemma 5 considers intervals that contain idle time.

Lemma 4 *There exists an optimal solution x^* to problem $\mathcal{P}_1$, where if $x_{il}^* = \Delta_l$ for some $i \in W$ and $l \in \bar{Q}$, then $l = \min\{\min\{j \in \bar{Q} \mid c_i(e_j - e_{j+1})/(s_j - s_{j+1}) \leq c_\alpha \hat{e}/\hat{s}\}, q\}$.*

Proof See the Online Appendix. $\square$

Lemma 5 *If x^* is an optimal solution to problem $\mathcal{P}_1(\omega)$ where $0 < x_{k\bar{q}}^* < \Delta_k$ and $k \neq \alpha$, then $c_k e_{\bar{q}}/s_{\bar{q}} \geq c_\alpha \hat{e}/\hat{s}$.*

Proof Suppose that $0 < x_{k\bar{q}}^* < \Delta_k$ and $k \neq \alpha$ for some optimal solution x^* to problem $\mathcal{P}_1(\omega)$. Consider the alternative solution

$$x_{ij}^0 = \begin{cases} x_{\alpha\beta}^* - \epsilon, & i = \alpha, j = \beta \\ x_{\alpha\beta+1}^* + \epsilon, & i = \alpha, j = \beta + 1 \\ x_{k\bar{q}}^* + \epsilon \hat{s}/s_{\bar{q}}, & i = k, j = \bar{q} \\ x_{ij}^*, & \text{otherwise,} \end{cases}$$

where $0 < \epsilon < \min\{x_{\alpha\beta}^*, (\Delta_k - x_{k\bar{q}}^*)s_{\bar{q}}/\hat{s}\}$. This solution is feasible because $x_{\alpha\beta}^0 + x_{\alpha\beta+1}^0 = x_{\alpha\beta}^* + x_{\alpha\beta+1}^*$ and $x_{k\bar{q}}^0 < \Delta_k$. There is no change in the total work processed since $\hat{s} = s_\beta - s_{\beta+1}$. Hence, $-\epsilon \hat{s} + \epsilon(\hat{s}/s_{\bar{q}})s_{\bar{q}} = 0$. The change in energy cost is $-\epsilon c_\alpha \hat{e} + \epsilon c_k e_{\bar{q}}(\hat{s}/s_{\bar{q}})$. Hence, x^* is not optimal if $c_k e_{\bar{q}}/s_{\bar{q}} < c_\alpha \hat{e}/\hat{s}$. $\square$

Recall that ω is defined as the last interval in which processing occurs. We now identify a dominant solution among those that complete in interval ω but strictly before time t_ω .

Lemma 6 *Suppose there exist multiple (α, β) pairs that generate solutions where $0 < x_{k\bar{q}}^* < \Delta_k$ for some $k \neq \alpha$ and satisfy Lemmas 2, 3 and 5. Among such solutions, the one which completes earliest has minimum cost.*

Proof Let (α_1, β_1) and (α_2, β_2) be two (α, β) pairs defined by (16) which satisfy the conditions of the lemma. Suppose that x' is the solution corresponding to (α_1, β_1) , x'' is the solution corresponding to (α_2, β_2) , and $x'_{k\bar{q}} < x''_{k\bar{q}}$. Hence, no processing speed is slower in any interval of x' than in x'' , and the speed is faster in at least one interval. Now, the definition of an (α, β) pair implies that $c_{\alpha_1}(e_{\beta_1} - e_{\beta_1+1})/(s_{\beta_1} - s_{\beta_1+1}) \geq c_{\alpha_2}(e_{\beta_2} - e_{\beta_2+1})/(s_{\beta_2} - s_{\beta_2+1})$. Therefore, $\gamma_{\alpha_1}(x'') > \beta_1$. Moreover, Lemma 5 establishes that

$$c_k \frac{e_{\bar{q}}}{s_{\bar{q}}} \geq c_{\alpha_1} \frac{e_{\beta_1} - e_{\beta_1+1}}{s_{\beta_1} - s_{\beta_1+1}} \geq c_{\alpha_2} \frac{e_{\gamma_{\alpha_1}(x'')} - e_{\gamma_{\alpha_1}(x'')+1}}{s_{\gamma_{\alpha_1}(x'')} - s_{\gamma_{\alpha_1}(x'')+1}}.$$

Consequently, increasing the speed of schedule x'' in interval α_1 reduces processing in interval k and does not increase cost. This process can be repeated for each interval i where $\gamma_i(x') < \gamma_i(x'')$. $\square$

The next result summarizes the optimality conditions given in Lemmas 1–6.

Theorem 3 *There exists an optimal solution x^* to Problem $\mathcal{P}_1(\omega)$ where*

- a. *If $\sum_{j \in \bar{Q}} x_{kj}^* = \Delta_k$ for $k \in W$, then $x_{kl}^* > 0$ where $l = \min\{j \in \bar{Q} \mid c_k(e_j - e_{j+1})/(s_j - s_{j+1}) \leq c_\alpha \hat{e}/\hat{s}\}$. If $k \neq \alpha$ or $0 < \sum_{j \in \bar{Q}} x_{ij}^* < \Delta_i$ for some $i \in W$, then $x_{kl}^* = \Delta_{kl}$. Otherwise, interval α may have $x_{\alpha\beta}^* > 0$ and $x_{\alpha\beta+1}^* > 0$, for $\beta \in \bar{Q}$.*
- b. *If $0 < \sum_{j \in Q} x_{kj}^* < \Delta_k$ for some $k \in W$, then $c_k e_{\bar{q}}/s_{\bar{q}} \geq c_\alpha \hat{e}/\hat{s}$. Further, $x_{k\bar{q}}^*$ is minimum among those feasible solutions that satisfy part a.*

Proof Part a follows from Lemmas 1–4. Part b follows from Lemmas 5 and 6. $\square$

4 Algorithms for the one-task problem

In order to consider the multiple task problem in Sect. 5, we first consider all possible starting and ending states of a single task. Idle time occurs before and after processing a task in four possible (start, end) configurations: (sleep, sleep), (sleep, powered-down), (powered-down, sleep), and (powered-down, powered-down). These are denoted as (ss) , (sp) , (ps) and (pp) , respectively. We use $\mathcal{P}^{ij}$, where $i, j \in \{s, p\}$ to describe the four corresponding cases of the one-task problem, and use $EC(ij)$ to describe the corresponding algorithms.

Based on the results in Sect. 3, we develop three procedures, which are combined in $EC(ij)$. The first procedure, justified by Theorem 2, eliminates (speed, energy) combinations that do not satisfy (15). The second procedure generates an initial schedule that has minimum total processing time. The last procedure examines all possible optimal schedules by systematically reducing the processing speeds of the schedule found in the second procedure.

The first procedure is used in all $EC(ij)$. In addition to removing dominated (speed, energy) combinations, this procedure also creates an ordered set G of (α, β) pairs to determine the order of interval speed reductions. This ordered set is used both in the second procedure to generate an initial schedule and in the third procedure to reduce systematically the processing speeds of the intervals.

Procedure InitG.

0. Input τ, c_i and t_i for $i \in T = \{1, \dots, \tau\}, q, e_j$ and s_j for $j \in Q = \{1, \dots, q\}$, and P .
1. If $P/s_1 > t_\tau$, then output “problem infeasible” and stop.
2. Set $\Delta_i = t_i - t_{i-1}$, for $i = 1, \dots, \tau$.

3. Reindex the elements of Q in nonincreasing s_j order. For $j = 2, \dots, q - 1$, if

$$\frac{e_{j-1} - e_j}{s_{j-1} - s_j} \leq \frac{e_j - e_{j+1}}{s_j - s_{j+1}},$$

then remove s_j and e_j , set $q = q - 1$, and reindex the remaining s_i and e_i accordingly.

4. For $i \in T$ and $j \in \{1, \dots, q - 1\}$, sort the $c_i(e_j - e_{j+1})/(s_j - s_{j+1})$ values in nonincreasing order. Let $G = \{(i, j)\}$, denote the corresponding ordered set of (i, j) pairs.
5. Output Δ_i for $i \in T = \{1, \dots, \tau\}$, q, e_j and s_j for $j \in Q = \{1, \dots, q\}$, and G .

Lemma 7 *Procedure InitG completes in $O(q\tau \log(\min\{q, \tau\}) + q \log q + \tau \log \tau)$ time.*

Proof Step 1 of Procedure InitG requires constant time and Step 2 requires $O(\tau)$ time. Creating an undominated order of speeds in Step 3 requires $O(q \log q)$ time. The interval costs are ordered in $O(\tau \log \tau)$ time. In Step 4, the ordered set G has $O(q\tau)$ entries, each taking constant time to generate. Since we can construct G either by merging the times or by merging the intervals, the time to generate G is $O(q\tau \log(\min\{q, \tau\}))$. Hence, the overall computation time required by InitG is $O(q\tau \log(\min\{q, \tau\}) + q \log q + \tau \log \tau)$. $\square$

While the first procedure is common to all $EC(ij)$, for $i, j \in \{\text{sleep, power-down}\}$, the second and third procedures require some modifications based on the particular configuration. We first consider Problem $\mathcal{P}^{sp}$. A procedure that finds an initial solution with minimal completion time is now provided. This schedule uses the fastest available processing speed, s_1 .

Procedure FirstInitSol

0. Input τ, c_i and t_i for $i \in \{1, \dots, \tau\}, q, e_j$ and s_j for $j \in Q = \{1, \dots, q\}, P$, and G .
1. Let ω denote the interval that contains time P/s_1 . Schedule at speed s_1 in $[0, P/s_1]$. Let x^0 denote the solution, z^0 its cost, and $d = t_\omega - P/s_1$. Let (α, β) , be the first element of G where $\alpha \leq \omega$. Remove (α, β) and all previous entries from G . Initialize the interval speed indices $\gamma_1 = \dots = \gamma_\omega = 1$. Output γ_i for $i \in T$, solution x^0 and cost z^0 , (α, β) , ω , d , and G , and stop.

The running time of Procedure FirstInitSol is bounded by the cardinality of G , i.e. $O(q\tau)$. We next describe a procedure to search through the set of potentially optimal solutions

for Problem $\mathcal{P}^{\beta\beta}$. The search proceeds by reducing processing speeds, which results in lengthening the schedule. This procedure is based on Theorem 3.

Procedure FirstUse

0. Input τ, c_i, γ_i , and Δ_i for $i \in T, q, e_j$ and s_j for $j \in Q$, $x^0, z^0, (\alpha, \beta), \omega, d$, and G .
Set $f = \Delta_\alpha$ and $\delta = 0$.
1. If $d \leq f(s_\beta - s_{\beta+1})/s_{\bar{q}}$, then set $f = f - ds_{\bar{q}}/(s_\beta - s_{\beta+1})$, $d = 0$, and go to Step 2.
Otherwise, set $\gamma_\alpha = \beta + 1$ and $d = d - f(s_\beta - s_{\beta+1})/s_{\bar{q}}$.
If $c_\omega e_{\bar{q}}/s_{\bar{q}} \geq c_\alpha \hat{e}/\hat{s}$ and $\delta = 0$, then set $\delta = 1$ and go to Step 2.
Otherwise, go to Step 3.
2. If the solution has cost less than z^0 , then update x^0 and z^0 .
If $d = 0$ and $\omega = \tau$, then output $x^* = x^0$ and $z^* = z^0$, and stop.
If $d = 0$, then set $\omega = \omega + 1$, $d = \Delta_\omega$, and $\delta = 0$, and go to Step 1.
3. If $G = \emptyset$, then output $x^* = x^0$ and $z^* = z^0$, and stop.
Otherwise, let (α, β) be the first element of G , where $\alpha \leq \omega$. Remove (α, β) and all prior entries from G . Set $f = \Delta_\alpha$. Go to Step 1.

Procedure FirstUse sets $\delta = 0$ for completion time interval ω , until a solution is found in $(t_{\omega-1}, t_\omega)$ that satisfies Condition b of Theorem 3. Then, the procedure sets $\delta = 1$ until a new schedule completion interval is considered. Step 1 generates all possible optimal solutions for interval ω : one that completes earliest in that interval, and one that completes at time t_ω . These solutions are evaluated in Step 2. The procedure terminates when t_τ is reached, or when $G = \emptyset$ which indicates that there are no further solutions to consider.

Lemma 8 Procedure FirstUse completes in $O(q\tau)$ time.

Proof Procedure FirstUse may consider every $(\alpha, \beta) \in G$. Each (α, β) combination generates at most two solutions, one in the interior and one at the end of an interval. Since at most $2|G|$ solutions may be considered, and it requires a constant number of operations to change from one solution to another, the computational work is $O(q\tau)$. $\square$

We now present an algorithm to solve Problem $\mathcal{P}^{\beta\beta}$.

Algorithm EC($\beta\beta$)

1. Call Procedure InitG.
2. Call Procedure FirstInitSol.
3. Call Procedure FirstUse.
4. Output x^* and z^* .

The correctness and computation time of this procedure are now established.

Theorem 4 Algorithm EC($\beta\beta$) finds an optimal solution for Problem $\mathcal{P}^{\beta\beta}$ in

$$O(q\tau \log(\min\{q, \tau\}) + q \log q + \tau \log \tau) \text{ time.}$$

Proof Procedure FirstInitSol finds the feasible solution with earliest completion time. From the discussion following Procedure FirstUse, the procedure considers all possible optimal solutions with larger completion times. Each interval contains at most two potentially optimal solutions that satisfy the conditions of Theorem 3. It follows that Algorithm EC($\beta\beta$) finds an optimal solution.

Regarding the running time of Algorithm EC($\beta\beta$), Lemma 7 establishes that Step 1 requires $O(q\tau \log(\min\{q, \tau\}) + q \log q + \tau \log \tau)$ time. Step 2 requires $O(q\tau)$ time. Further, Lemma 8 shows that Step 3 requires $O(q\tau)$ time. As a result, the time complexity of Algorithm EC($\beta\beta$) is $O(q\tau \log(\min\{q, \tau\}) + q \log q + \tau \log \tau)$. $\square$

In Problem $\mathcal{P}^{\beta\beta}$, once the task completes, the machine sleeps until time t_τ . Observe that if the time horizon is reversed for Problem $\mathcal{P}^{\beta\beta}$, then the problem is equivalent to Problem $\mathcal{P}^{\beta\beta}$. This suggests Procedures FirstInitSolR and FirstUseR, which are equivalent to Procedures FirstInitSol and FirstUse, respectively, except that the schedules are initiated from time t_τ and built backwards toward time 0. Hence, the following procedure finds an optimal solution to Problem $\mathcal{P}^{\beta\beta}$.

Algorithm EC($\beta\beta$): Identical to Algorithm EC($\beta\beta$), where Procedures FirstInitSol and FirstUse are replaced by Procedures FirstInitSolR and FirstUseR, respectively.

The correctness and computation time of this procedure are now established.

Theorem 5 Algorithm EC($\beta\beta$) finds an optimal solution for problem $\mathcal{P}^{\beta\beta}$ in

$$O(q\tau \log(\min\{q, \tau\}) + q \log q + \tau \log \tau) \text{ time.}$$

Proof The proof of optimality follows from Theorem 4. Since the complexity of all procedures is the same, the computation time also follows from Theorem 4. $\square$

We next consider Problem $\mathcal{P}^{\beta\beta}$. The following result shows that there exists an optimal schedule where either the starting interval has a power-up or the ending interval has a power-down.

Lemma 9 For problem $\mathcal{P}^{\beta\beta}$, there exists an optimal solution x^* where the starting interval $\lambda \in T$ and ending interval $\omega \in T$ satisfy $x_{\lambda q+1}^* = 0$ if $c_\lambda \leq c_\omega$, and $x_{\omega q+1}^* = 0$ if $c_\lambda > c_\omega$. Further, $x_{\lambda q}^* = 0$ and $x_{\omega q}^* = 0$.

Proof If $x_{\lambda q+1}^* > 0$ and $x_{\omega q+1}^* > 0$, then starting the schedule earlier or later by a small amount does not affect the processing in intervals $\lambda + 1, \dots, \omega - 1$. If $c_\lambda \leq c_\omega$, then starting earlier to fill interval λ does not increase the cost

of the schedule. In this case, processing begins at $t_{\lambda-1}$ and the machine powers down somewhere in interval ω . Having $x_{\lambda q}^* > 0$ or $x_{\omega q}^* > 0$ increases the cost of the schedule without any benefit. Alternatively, if $c_\lambda > c_\omega$, then ending later to fill interval ω does not increase the cost of the schedule. $\square$

We now present a procedure that finds an optimal solution for Problem $\mathcal{P}^{pp}$.

Algorithm EC($\mathcal{P}^{pp}$):

1. Call Procedure InitG.
2. Let $\bar{\lambda}$ be the interval that contains time $t_\tau - P/s_1$. Set $t_0 = 0$ and $\lambda = 1$.
Let ω be the interval that contains time $t_{\lambda-1} + P/s_1$.
3. If $c_\lambda \leq c_\omega$, then call Procedures FirstInitSol and FirstUse, where τ is set to ω .
Otherwise, call Procedures FirstInitSolR and FirstUseR, where τ is set to ω .
Among all solutions considered thus far, let x^0 denote the best solution and z^0 its cost.
4. If $\omega < \tau$, then set $\omega = \omega + 1$, update G , and go to Step 3.
Otherwise, set $\lambda = \lambda + 1$.
If $\lambda \leq \bar{\lambda}$, then update G and go to Step 2.
Otherwise, output z^0 and x^0 , and stop.

For any start interval $\lambda \in \{1, \dots, \bar{\lambda}\}$, Step 2 of Algorithm EC($\mathcal{P}^{pp}$) finds the earliest possible completion interval. Step 3 calls either Procedures FirstInitSol and FirstUse, or Procedures FirstInitSolR and FirstUseR, depending on the condition in Lemma 9. When either ω or λ is changed, G is updated by adding or deleting the relevant (α, β) pairs.

Theorem 6 Algorithm EC($\mathcal{P}^{pp}$) finds an optimal solution for problem $\mathcal{P}^{pp}$ in $O(q\tau^3 + q \log q)$ time.

Proof Algorithm EC($\mathcal{P}^{pp}$) considers all possible power-up intervals λ and power-down intervals ω . Since all such schedules are considered, Theorems 3 and 4 are applicable.

Calling Procedure InitG requires $O(q\tau \log(\min\{q, \tau\}) + q \log q + \tau \log \tau)$ time (see Lemma 7). Procedures FirstInitSol, FirstUse, FirstInitSolR, and FirstUseR can be called $O(\tau^2)$ times. The updates to G in Step 4 are performed by adding and/or removing each (α, β) pair at most once. Thus, it follows from Lemma 8 that the overall computational requirement of Algorithm EC($\mathcal{P}^{pp}$) is $O(q\tau^3 + q \log q)$ time. $\square$

For problems $\mathcal{P}^{\beta p}$, $\mathcal{P}^{p\beta}$, and $\mathcal{P}^{pp}$, the first interval without processing is the new interval that is chosen for processing. This is true for both the initial solution and solution improvement.

Problem $\mathcal{P}^{\beta\beta}$ requires a slightly different strategy. When choosing a new interval for processing, the interval $i \in T$

that has the minimum c_i is selected. With this exception, the procedures for finding an initial solution and for solution improvement are identical to those in Procedures FirstInitSol and FirstUse, respectively.

5 Problem $\mathcal{P}$

When there are multiple tasks and release dates, continuous processing may no longer be desired. For example, it may be optimal to complete a task before the next task is released. Thus, finding an optimal solution is a combinatorial problem regarding which tasks should be processed between idle periods. This implies that while Problem $\mathcal{P}$ can still be solved by a series of linear programs, the number grows exponentially with the number of tasks. Nevertheless, we are able to develop an optimal strongly polynomial time solution procedure, which is called Algorithm EnergyOpt. The approach constructs $O(n^2)$ groupings of adjacent tasks, which we define as *blocks*. Each block represents a node in a graph, and powering down is only allowed between blocks. The nodes are connected so that a path through the graph represents a potential optimal schedule. A shortest path in the graph provides a minimum cost schedule.

To allow for greater generality, we assume that the machine is initially powered down. Hence, processing is not required to start at time zero even when tasks are available at this time.

The first step is to construct sequences of adjacent tasks with the following properties.

Definition of Block $B_{ij}(h_1, h_2)$:

1. Set of tasks $h_1, \dots, h_2 \subseteq N$, where $h_1 \leq h_2$.
2. Processing of the tasks in the block starts no earlier than time r_{h_1} and completes no later than time $\min\{r_{h_2+1}, t_\tau\}$.
3. The completion time of task $h_k \in \{h_1, \dots, h_2 - 1\}$ is no earlier than r_{h_k+1} .
4. The starting machine state is $i \in \{\beta, p\}$, and the ending machine state is $j \in \{\beta, p\}$.

The definition of a block $B_{ij}(h_1, h_2)$ implies that while it may contain periods of sleep, there is always a task that is available for processing. Hence, no power-down is possible from the start of processing task h_1 until the completion of task h_2 . Observe that a task is a special case of a block with a single element. If two blocks are combined into one larger block, the ending state of the first block and the starting state of the second block must be β . Otherwise, there is a power-down and power-up in the middle of the block. Further, the reason for combining the two blocks and not keeping the two blocks separate is that some task processing that is performed in one of the blocks now is performed more cheaply in some

of the time intervals of the other block. Since tasks cannot be processed before their release dates, this implies that in a combined block, some of the task processing in the first block is now processed during time intervals of the second block.

Some additional notation is needed. The tasks of the combined block must be consecutively indexed. Hence, when we combine two blocks, they must have the form $B_{ij}(h_1, h_2)$ and $B_{kl}(h_2 + 1, h_3)$, for $i, l \in \{\mathcal{J}, \mathcal{P}\}$ and $j = k = \mathcal{J}$. Let $B_{ij}(h_1, h_2)$ be block 1 and $B_{kl}(h_2 + 1, h_3)$ be block 2. Associated with blocks 1 and 2 are minimum cost schedules x^1 and x^2 and energy costs $z(x^1)$ and $z(x^2)$, respectively. Also, let $p^1 = \sum_{k=h_1}^{h_2} p_k$, $p^2 = \sum_{k=h_2+1}^{h_3} p_k$, $r^1 = r_{h_1}$, and $r^2 = r_{h_2+1}$. Finally, for $k \in \{1, 2\}$, schedule x^k has $(\alpha^k, \beta^k) \equiv (\alpha(x^k), \beta(x^k))$ as the final (α, β) pair and associated (α, β) list G^k , $\hat{e}^k = e_{\beta^k} - e_{\beta^k+1}$ and $\hat{s}^k = s_{\beta^k} - s_{\beta^k+1}$, and a set of intervals J^k used by block k .

The combined block has the same properties as any other block and must satisfy the conditions of Theorem 3. In particular, the combined block must have an α and β that satisfy the conditions of the theorem. Suppose that

$$c_{\alpha^1} \frac{\hat{e}^1}{\hat{s}^1} < c_{\alpha^2} \frac{\hat{e}^2}{\hat{s}^2}. \quad (17)$$

Then, it is more costly to process tasks in block 2 than in block 1. As a result, it is not optimal to delay some of the processing of tasks in block 1 to block 2. This means that there can be no benefit by combining the two blocks. Hence, the requirement that (17) is violated is a condition for combining two blocks.

We now develop a procedure to concatenate two consecutive blocks into a single block. This procedure is repeated to create blocks with more tasks. There are four possible (start, end) condition combinations for the combined block. These combinations are $(\mathcal{J}, \mathcal{J})$, $(\mathcal{J}, \mathcal{P})$, $(\mathcal{P}, \mathcal{J})$ and $(\mathcal{P}, \mathcal{P})$. Each of these combinations requires a different procedure for combining the two blocks. These four procedures are designated as Procedure Combine(ij), where $i, j \in \{\mathcal{J}, \mathcal{P}\}$ are the starting state of block 1 and the ending state of block 2, respectively. In each of the procedures, Step 1 creates an initial schedule based on (α^1, β^1) . Then, Step 2 tries to improve the solution by reducing interval processing speeds.

Procedure Combine($\mathcal{J}\mathcal{J}$).

- For blocks 1 and 2, input c_i and Δ_i for $i \in \{\rho_{h_1}, \dots, \rho_{h_3+1} - 1\}$; q, e_j and s_j for $j \in Q$; the optimal block 1 solution with the current (α^1, β^1) , and (α^2, β^2) and J^2 . If $c_{\alpha^1} \hat{e}^1 / \hat{s}^1 < c_{\alpha^2} \hat{e}^2 / \hat{s}^2$, then output block $B_{\mathcal{J}\mathcal{J}}(h_1, h_3)$ is not part of an optimal solution, and return. Set $(\alpha, \beta) = (\alpha^1, \beta^1)$ and $i = \rho_{h_2+1}$.
- Set $\gamma_i = \min\{\min\{j \in \bar{Q} \mid c_i(e_j - e_{j+1}) / (s_j - s_{j+1}) \leq c_{\alpha} \hat{e} / \hat{s}\}, q\}$. Remove entries $(i, 1), \dots, (i, \gamma_i - 1)$ from G .

Set $i = i + 1$, and repeat Step 1 until the total work processed in intervals $\rho_{h_2+1}, \dots, \omega \leq \rho_{h_3+1} - 1$ is p^2 .

Let d denote the amount of idle time in interval ω .

- Call Procedure FirstUse, as described for $\mathcal{P}^{\mathcal{J}\mathcal{J}}$ in the last paragraph of Sect. 4.
- Output x^* , current (α, β) , and z^* .

Procedure Combine($\mathcal{J}\mathcal{P}$)

- For blocks 1 and 2, input c_i , and Δ_i for $i \in \{\rho_{h_1}, \dots, \rho_{h_3+1} - 1\}$; q, e_j and s_j for $j \in Q$; the optimal block 1 solution with the current (α^1, β^1) , and (α^2, β^2) and J^2 . If $c_{\alpha^1} \hat{e}^1 / \hat{s}^1 < c_{\alpha^2} \hat{e}^2 / \hat{s}^2$, then output block $B_{\mathcal{J}\mathcal{P}}(h_1, h_3)$ is not part of an optimal solution, and return. Set $(\alpha, \beta) = (\alpha^1, \beta^1)$ and $i = \rho_{h_2+1}$.
- Set $\gamma_i = \min\{\min\{j \in \bar{Q} \mid c_i(e_j - e_{j+1}) / (s_j - s_{j+1}) \leq c_{\alpha} \hat{e} / \hat{s}\}, q\}$. Remove entries $(i, 1), \dots, (i, \gamma_i - 1)$ from G . Set $i = i + 1$, and repeat Step 1 until the total work processed in intervals $\rho_{h_2+1}, \dots, \omega \leq \rho_{h_3+1} - 1$ is p^2 . Let d denote the amount of idle time in interval ω .
- Call Procedure FirstUse.
- Output x^* , current (α, β) , and z^* .

Procedure Combine($\mathcal{P}\mathcal{J}$): Solve for intervals $\rho_{h_1}, \dots, \rho_{h_3+1} - 1$ using Algorithm EC($\mathcal{P}\mathcal{J}$), where in addition to stopping if $\omega < 0$ or $G = \emptyset$, Procedure FirstUseR also stops when

$\sum_{\rho_h \leq i \leq \rho_{h_3+1}-1} \sum_{j \in \bar{Q}} s_j x_{ij} = \sum_{k=h}^{\rho_{h_3+1}-1} p_k$ for task $h \in \{\rho_{h_1}, \dots, \rho_{h_3+1} - 1\}$, and the speed in interval ρ_h is being reduced.

If $\rho_h = \rho_{h_1}$, then output x^* , the current (α, β) , and z^* , and stop.

Otherwise, call Procedure FirstUseR with the current solution for intervals $\rho_{h_1}, \dots, \rho_h - 1$.

Procedure Combine($\mathcal{P}\mathcal{P}$): Solve for intervals $\rho_{h_1}, \dots, \rho_{h_3+1} - 1$ using Algorithm EC($\mathcal{P}\mathcal{P}$), where Procedure FirstUse is replaced by Procedure Combine($\mathcal{J}\mathcal{P}$) and Procedure FirstUseR is replaced by Procedure Combine($\mathcal{P}\mathcal{J}$).

Procedure Combine($\mathcal{P}\mathcal{J}$) has an additional stopping criterion. Initially, tasks are processed as late as possible. By reducing interval speeds, task processing occurs in earlier intervals. The procedure must satisfy the requirement that it does not start processing a task before its release date. When this occurs, further processing speed reduction is not possible.

Similar to Algorithm EC($\mathcal{P}, \mathcal{P}$), Procedure Combine($\mathcal{P}\mathcal{P}$) must consider each possible start and end interval to find the best $(\mathcal{P}\mathcal{P})$ schedule.

The following result establishes the correctness of the above procedures.

Theorem 7 Given optimal blocks $B_{i\beta}(h_1, h)$ and $B_{\beta j}(h + 1, h_2)$, Procedure Combine(ij) finds an optimal schedule for block $B_{ij}(h_1, h_2)$, for $i, j \in \{\beta, \rho\}$, and $1 \leq h_1 \leq h \leq h_2 \leq n$, where $h_1 < h_2$.

Proof Procedure Combine($\beta\beta$) is similar to Algorithm EC($\beta\beta$). If the speeds in block 2 can be adjusted so that the speeds and (α^1, β^1) satisfy Theorem 3, then some processing of p^1 can occur after time r^2 . Hence, Procedure Combine($\beta\beta$) finds an optimal schedule. Further, if it is not possible to process all the p^2 units of work in block 2, then cost reduction can occur only if some processing of block 2 occurs before r^2 , which is impossible. In that case, Step 1 of Procedure Combine($\beta\beta$) outputs that no combined block consisting of blocks 1 and 2 need be considered in the search for an optimal schedule.

For Procedure Combine($\beta\rho$), Step 1 sets the speeds of block 2 according to Theorem 3. Consequently, Procedure FirstUse can search for an optimal solution for the combination of blocks 1 and 2, given that block 1 now completes at r^2 . We let ω^2 denote the last interval in block 2 that contains processing. If $c_{\alpha^1}\hat{e}^1/\hat{s}^1 \leq \min\{c_{\alpha^2}\hat{e}^2/\hat{s}^2, c_{\omega^2}e\bar{q}/s\bar{q}\}$, then cost reduction can occur only if some processing of block 2 occurs before r^2 , which is impossible. In that case, Step 1 of Procedure Combine($\beta\rho$) outputs that no combined block consisting of blocks 1 and 2 need be considered further.

For Procedure Combine($\rho\beta$), Theorem 5 establishes that an optimal solution is found for the combined block. The restriction that Procedure FirstUseR stops when

$\sum_{\rho_h \leq i \leq \rho_{h_3+1}-1} \sum_{j \in \bar{Q}} s_j x_{ij} = \sum_{k=h}^{\rho_{h_3+1}-1} p_k$ for task $h \in \{\rho_{h_1}, \dots, \rho_{h_3+1}-1\}$ ensures that the definition of a block is not violated.

For Procedure Combine($\rho\rho$), Theorem 6 and the fact that Procedure Combine($\rho\beta$) and Procedure Combine($\beta\rho$) find optimal solutions, establish that an optimal solution is found for the combined block. $\square$

Identifying all possible optimal solutions requires the generation of all feasible blocks. A procedure to accomplish this is now presented.

Procedure BlockGen

1. Call Algorithm EC(ij), for $i, j \in \{\beta, \rho\}$, $1 \leq h \leq n$.
2. For $i, j \in \{\beta, \rho\}$:
For $h_2 = 2, \dots, n$:
For $h_1 = 1, \dots, h_2 - 1$:
Find the smallest $h \in \{h_1, \dots, h_2\}$ such that $B_{i\beta}(h_1, h)$ and $B_{\beta j}(h + 1, h_2)$ exist. Call Procedure Combine(ij) for $B_{i\beta}(h_1, h)$ and $B_{\beta j}(h + 1, h_2)$.
If h does not exist, then block $B_{ij}(h_1, h_2)$ does not exist.
3. Output all feasible combined blocks $B_{ij}(h_1, h_2)$ and their costs $z^*(B_{ij}(h_1, h_2))$, for $i, j \in \{\beta, \rho\}$, $1 \leq h_1 \leq h_2 \leq n$. Stop.

Lemma 10 Procedure BlockGen generates all blocks of potentially optimal subsequences of tasks that do not contain a power-down in $O(n^2(q\tau^3 + q \log q))$ time.

Proof Step 2 considers all possible starting and ending conditions and all possible task subsequences. Theorem 7 establishes that an optimal block $B_{ij}(h_1, h_2)$ is constructed from optimal blocks $B_{i\beta}(h_1, h)$ and $B_{\beta j}(h + 1, h_2)$ for any $h \in \{h_1, \dots, h_2\}$. As a result, all possible optimal blocks are constructed.

The running time follows from Theorems 4, 5 and 6, and the application of each procedure $O(n^2)$ times in Step 2. $\square$

A feasible schedule for problem $\mathcal{P}$ is formed by concatenation of suitably selected blocks from among those found by Procedure BlockGen. We define a graph, $\mathcal{G}$, where the nodes represent the blocks, and the arcs specify the cost of concatenating the two blocks associated with the arc. An optimal schedule is represented by the shortest path from start node $\mathcal{S}$ to terminal node $\mathcal{T}$. This schedule is a concatenation of the individual block schedules, each of which contains one or more tasks. We use the following algorithm to construct $\mathcal{G}$.

Algorithm Build $\mathcal{G}$

0. Input the power-up cost K .
1. Call Procedure BlockGen.
2. Construct a node for each feasible block found by Procedure BlockGen, an additional dummy source node $\mathcal{S}$, and an additional dummy sink node $\mathcal{T}$.
3. Connect node $\mathcal{S}$ to each node that represents a block $B_{\rho j}(1, \cdot)$, by a directed arc with cost $K + z^*(B_{\rho j}(1, \cdot))$, for $j \in \{\beta, \rho\}$.
4. Connect each node that represents a block $B_{i\rho}(\cdot, n)$ to node $\mathcal{T}$, by a directed arc with cost 0, for $i \in \{\beta, \rho\}$.
5. For $1 \leq h_1 < h_2 < h_3 \leq n$, connect block $B_{ij}(h_1, h_2)$ to block $B_{jk}(h_2 + 1, h_3)$ by an arc with cost
 - a. $K + z^*(B_{jk}(h_2 + 1, h_3))$ if $j = \rho$.
 - b. $z^*(B_{jk}(h_2 + 1, h_3))$ if $j = \beta$.
6. Output graph $\mathcal{G}$.

Since the tasks are processed in index order and the initial state of the machine is powered down, Step 3 requires that task 1 is processed first after the machine is powered up. Step 4 provides a symmetric requirement for task n at the end of the schedule. Step 5 specifies the cost of concatenating two blocks. Depending on the types of blocks being concatenated, this cost may include a power-up cost. The following result establishes that the graph constructed by Algorithm Build $\mathcal{G}$ can be used to find an optimal schedule.

Theorem 8 *Each feasible path from S to T in graph $\mathcal{G}$ corresponds to a feasible schedule for problem $\mathcal{P}$. Moreover, the shortest feasible path corresponds to an optimal schedule.*

Proof We first show that every feasible path from S to T in $\mathcal{G}$ corresponds to a feasible schedule for problem $\mathcal{P}$. Since Algorithm Build $\mathcal{G}$ connects block $B_{ij}(h_1, h_2)$ by an arc to block $B_{jk}(h_2 + 1, h_3)$, a path from S to T includes each task exactly once, in the order $h_1, \dots, h_n$. Power-downs are not allowed within a block. A requirement of block $B_{ij}(h_1, h_2)$ is that it completes by time r_{h_2+1} . As a result, a power-down never occurs when tasks are available for processing. Further, two blocks $B_{ij}(h_1, h_2)$ and $B_{kl}(h_2 + 1, h_3)$ are connected by an arc only when $j = k \in \{\mathcal{J}, \mathcal{P}\}$. Hence, either the first block powers down after completion and the second block powers up at the start of processing, or the first block goes to sleep at completion and the second block starts with the machine already powered-up. As a result, every path is associated with a feasible schedule.

To complete the proof, we show that Algorithm Build $\mathcal{G}$ compares the cost of all feasible schedules. The procedure constructs a node for every feasible block of tasks. Consequently, the set of paths represents all possible power-up and power-down combinations between tasks. Further, between a power-up and the next power-down, an optimal schedule for a block or blocks separated by sleep is found (see Theorems 4–7). Thus, a minimum cost schedule is generated for every possible combination of power-ups and power-downs. Since one of those combinations occurs in an optimal schedule, one of the paths in $\mathcal{G}$ corresponds to an optimal schedule. Finally, because the path and associated schedule costs are equal by construction, a shortest path in $\mathcal{G}$ corresponds to an optimal schedule. Algorithm EnergyOpt finds an optimal solution to our problem by calling Algorithm Build $\mathcal{G}$, and then solving the resulting graph by dynamic programming. We now present our main result for problem $\mathcal{P}$. $\square$

Theorem 9 *Algorithm EnergyOpt finds an optimal solution for problem $\mathcal{P}$ which corresponds to a shortest path from node S to node T in graph $\mathcal{G}$, and requires $O(n^2 \max(\{q\tau^3 + q \log q, n^2\}))$ time.*

Proof The optimality of the solution found for problem $\mathcal{P}$ follows from Lemma 10, Theorem 8 and the definition of a shortest path.

Regarding the computation time, it follows from Lemma 10 that the generation of all potentially optimal blocks requires $O(n^2(q\tau^3 + q \log q))$ time. Since graph $\mathcal{G}$ contains $O(n^2)$ nodes, the application of a simple shortest path algorithm requires $O(n^4)$ time (Ahuja et al., 1993). Therefore, the overall computational work to solve problem $\mathcal{P}$ requires $O(n^2 \max(\{q\tau^3 + q \log q, n^2\}))$ time. $\square$

Our main result summarized in Theorem 9 shows that, although problem $\mathcal{P}$ is not a convex optimization problem

even with only one task (see Example 1), the general problem with n tasks can be solved by a strongly polynomial time algorithm.

6 Computational study

In this section, we compare the computational performance of Algorithm EnergyOpt with that of a benchmark CPLEX optimizer. Our algorithm, EnergyOpt, is coded in C++ and compiled using the Visual Studio C++ 2022 compiler. The integer program ($\mathcal{P}$) is solved using IBM ILOG CPLEX. Computational testing is performed on an HP computer with an Intel Core I5-4590 CPU that runs at 3.30 GHz, and with 16GB of installed RAM.

All generated data in our experiments are integer. The length of the planning horizon is set to $t_\tau = 10,000$. Selected by the experimental design (see Table 1), the number of intervals $\tau \in \{20, 100, 200\}$. We randomly select without replacement $\tau - 1$ integers from $[1, 9,999]$. Given t_τ , these numbers are sorted in increasing order to generate t_i for $i \in T$. The unit energy cost of interval $i \in T$ is generated from the Integer Uniform distribution $UI[1, 20]$. The machine speed, s_j , and energy usage, e_j for $j \in Q$, are selected randomly without replacement from $[1, 15]$ and $[1, 30]$, respectively. The number of speed–energy pairs $q \in \{3, 6, 10\}$. The number of tasks $n \in \{10, 50, 100\}$. The nominal task processing time is generated from $UI[1, \hat{p}]$, where $\hat{p} \in \{800, 1600, 8000\}$. The value of $\hat{p}$ is selected based on the number of tasks to achieve an expected utilization rate $\in \{25\%, 50\%, 75\%\}$ (see below). The release dates are n integers randomly selected without replacement over the interval $[1, 9999]$. If a generated data set is infeasible because all jobs cannot be completed by t_τ even at speed s_1 , then the data set is discarded and regenerated. The power-up cost $\in \{50, 1000, 5000\}$.

Each row of Table 1 provides summary statistics of CPU times for 10 instances of the problem with the various characteristics described. The first five columns show, respectively: the number of intervals, the number of tasks, the power-up cost, the number of machine speeds, and the utilization factor. These characteristics are chosen as those most relevant to CPU time performance, following Hall and Posner (2001). The utilization factor equals the work required divided by the processing capacity, i.e. $n \times$ average task processing time / $t_\tau \times$ average speed. In Columns 6–8, we report the mean and maximum CPU time in seconds, as well as the number out of 10 instances solved by CPLEX within 200 CPU seconds. Columns 9 and 10 similarly show the mean and maximum CPU time in seconds over 10 instances for Algorithm EnergyOpt.

The first row in Table 1 is our base case. Each pair of subsequent rows varies one parameter around the base case

Table 1 CPU Times for CPLEX and EnergyOpt

τ	n	Power-up		Utilization (%)	CPLEX			EnergyOpt	
		Cost	q		Average (CPU seconds)	Max	Number Solved	Average (CPU seconds)	Max
100	50	1000	6	50	122.418	200	5	0.019	0.031
20	50	1000	6	50	3.217	6.488	10	0.005	0.007
200	50	1000	6	50	185.896	200	2	0.083	0.181
100	10	1000	6	50	17.056	32.035	10	0.003	0.006
100	100	1000	6	50	181.487	200	2	0.087	0.128
100	50	50	6	50	192.682	200	1	0.021	0.035
100	50	5000	6	50	56.210	200	9	0.020	0.027
100	50	1000	3	50	100.569	200	8	0.018	0.037
100	50	1000	10	50	130.419	200	5	0.021	0.037
100	50	1000	6	25	75.019	200	9	0.032	0.040
100	50	1000	6	75	95.619	200	7	0.011	0.030

Table 2 Large Instance CPU Times for EnergyOpt

τ	n	EnergyOpt	
		Average (CPU seconds)	Max (CPU seconds)
500	50	1.00	2.07
100	200	0.51	1.26
500	200	7.11	10.23

Table 3 Regression results for EnergyOpt

Variable	Coefficient	Std. error	t stat	p value
Intercept	0.1114	0.3009	0.3702	0.7118
$n^2\tau^3/10^4$	1.32E-08	4.57E-10	28.7943	5.28E-59
$n^4/10^4$	2.23E-06	1.03E-06	2.1684	0.0319
C	-9.9E-06	3.91E-05	-0.2534	0.8003
q	-0.0018	0.0314	-0.0556	0.9557
U	-0.0004	0.0044	-0.0937	0.9255

value. For example, rows 2 and 3 study the effect of varying the number of intervals, τ , from 20 to 200, around the base case value of 100. The subsequent pairs of rows study the effect on CPU time of changing the number of tasks, the power-up cost, the number of machine speeds, and the utilization factor, respectively.

While CPLEX could not solve larger problem instances within 600 seconds, presented in Table 2 are larger instances for Energy Opt. In this table, Power-up Cost = 1000, $q = 6$, and Utilization = 50%. We report the mean and maximum CPU times over 10 instances. Even for much larger instances than previously solved *heuristically* by Shrouf et al. (2014), EnergyOpt finds *optimal* solutions quickly.

The results in Tables 1 and 2 support Theorem 9 regarding the approximate running time of EnergyOpt. To provide details of the exact relationship, we develop a multiple non-linear regression model to estimate the CPU time required by the algorithm. Based on the running time in Theorem 9, we first choose independent variables $n^2\tau^3$ and n^4 . Other potentially relevant variables include the power-up cost denoted by C , the number of machine speeds q , and the utilization rate denoted by U . As a result, the model is

$$CPU = \beta_0 + \beta_1 n^2 \tau^3 / 10000 + \beta_2 n^4 / 10000 + \beta_3 C + \beta_4 q + \beta_5 U + \epsilon, \quad (18)$$

where $\beta_0, \dots, \beta_5$ are regression coefficients to be estimated from the 140 observations for the algorithm, and ϵ is a residual error term. We divide the second and third terms in (18) by 10,000 to prevent small regression coefficients. The results appear in Table 3 and the following discussion.

The regression model achieves a Multiple R value of 0.961, an R Square value of 0.924 and an Adjusted R Square value of 0.922, indicating a high level of explanation of the CPU time. The results in Table 3 show that the number of intervals (τ) and the number of tasks (n) have significant positive effects on CPU time, in accordance with Theorem 9. These results are consistent with the fact that τ and n are the main determinants of the model size.

7 Generalizations of problem $\mathcal{P}$

Our problem is close to the boundary of problems that are polynomially solvable and those that are NP -complete. Many generalizations of problem $\mathcal{P}$ are intractable. Two examples discussed by Garey and Johnson (1979) are problems with multiple parallel machines and problems with task deadlines. We now consider two other generalizations that are close to

problem $\mathcal{P}$. The proofs of the following two results appear in the Appendix.

The first generalization, which we denote by problem $\mathcal{P}^S$, allows machine speed changes to occur only at the start of a task.

Theorem 10 *The recognition version of problem $\mathcal{P}^S$ is strongly NP-complete.*

The second generalization, which we denote by problem $\mathcal{P}^D$, allows the machine to power down any time a task completes, even if another task is available.

Theorem 11 *The recognition version of problem $\mathcal{P}^D$ is strongly NP-complete.*

8 Concluding remarks

Motivated by the high cost of energy in both manufacturing and computer processing industries, this paper extends the literature of scheduling with energy considerations. Companies can now benchmark their current energy usage policy against an optimal one, which enables them to identify opportunities for cost reduction. Also, as Smartgrid implementations become more prevalent, this can provide a competitive advantage to those companies that optimize their energy costs. A further contribution of our work is that the environmental impact of a company's manufacturing operations can be reduced by more energy-efficient scheduling. In most countries, baseload power is provided by the cleanest generators. Only when demand is high are the older and less-efficient generators switched on (Wright, 2013). In this context, reducing demand at peak times can significantly reduce emissions by taking the dirtiest plants offline. Also, our results may motivate companies to obtain precise data about their energy usage rates and costs.

Several interesting problems remain open for further study. Our models assume that processing of a task cannot be interrupted by the processing of a different task. However, depending on the task energy cost structure, it may be advantageous to allow such interruptions. It would also be worthwhile to consider problems where each task has its own deadline. Another research area models demand that is unknown prior to arrival. Online as well as probabilistic models can be considered. Other scheduling cost functions can also be studied. Examples include problems that have costs based on the task delivery time. Also, while we assume continuous availability of processing resources, there are various practical considerations such as maintenance time which may prevent this. Alternatively, there may be situations where a company may want to include side constraints, such as CO₂ emission restrictions. Also, it would be interesting to study tradeoffs in minimizing energy cost between the use of conventional and green energy, following the work of Goiri et

al. (2011). Finally, models could integrate energy scheduling with other production decisions such as inventory storage. In conclusion, there are many related open problems, and we hope that our work will encourage further study of this important research area.

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s10951-026-00872-5>.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network flows: Theory, algorithms, and applications*. Englewood Cliffs: Prentice Hall.
- Albers, S. (2010). Energy-efficient algorithms. *Communications of the ACM*, 53(5), 88–96.
- Antoniadis, A., Kling, P., Ott, S., & Riechers, S. (2017). Continuous speed scaling with variability: A simple and direct approach. *Theoretical Computer Science*, 678, 1–13.
- Cao, Z., Wang, Z., Zhang, Y., & Liu, S. (2006). On several scheduling problems with rejection or discretely compressible processing times. In: J.-Y. Cai, S. B. Cooper, & A. Li (Eds.), *Theory and applications of models of computation. TAMC 2006. Lecture notes in computer science* (vol. 3959, pp. 90–98). Berlin, Heidelberg: Springer.
- Carlucci, D., Renna, P., & Materi, S. (2023). A job-shop scheduling decision-making model for sustainable production planning with power constraint. *IEEE Transactions on Engineering Management*, 70(5), 1923–1933.
- Catanzaro, D., Pesenti, R., & Ronco, R. (2023). Job scheduling under time-of-use energy tariffs for sustainable manufacturing: A survey. *European Journal of Operational Research*, 308, 1091–1109.
- Chen, Z.-L. (2004). Simultaneous job scheduling and resource allocation on parallel machines. *Annals of Operations Research*, 129, 139–153.
- Chen, Z.-L., Lu, Q., & Tang, G. (1997). Single machine scheduling with discretely controllable processing times. *Operations Research Letters*, 21, 69–76.
- Daniels, R. L., Hoopes, B. J., & Mazzola, J. B. (1996). Scheduling parallel manufacturing cells with resource flexibility. *Management Science*, 42, 1260–1276.
- Daniels, R. L., Hoopes, B. J., & Mazzola, J. B. (1997). An analysis of heuristics for the parallel-machine flexible-resource scheduling problem. *Annals of Operations Research*, 70, 439–472.
- Daniels, R. L., & Mazzola, J. B. (1994). Flow shop scheduling with resource flexibility. *Operations Research*, 42, 504–522.

- Eurostat. (2024). Final energy consumption in industry—Detailed statistics. Available at: https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Final_energy_consumption_in_industry_-_detailed_statistics. Accessed 28 November 2025
- Fang, K., Uhan, N., Zhao, F., & Sutherland, J. W. (2011). A new shop scheduling approach in support of sustainable manufacturing. In J. Hesselbach & C. Herrmann (Eds.), *Glocalized solutions for sustainability in manufacturing, proceedings of the 18th CIRP international conference on life cycle engineering*. Springer.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. San Francisco: Freeman.
- Giglio, D., Paolucci, M., & Roshani, A. (2017). Integrated lot sizing and energy-efficient job shop scheduling problem in manufacturing/remanufacturing systems. *Journal of Cleaner Production*, 148, 624–641.
- Goiri, I., Le, K., Haque, M. E., Beauchea, R., Nguyen, T. D., Guitart, J., Torres, J., & Bianchini, R. (2011). GreenSlot: Scheduling energy consumption in green datacenters. In *SC '11, Seattle, WA, November 12–18*.
- Grigoriev, A., Sviridenko, M., & Uetz, M. (2007). Machine scheduling with resource dependent processing times. *Mathematical Programming*, 110, 209–228.
- Grigoriev, A., & Uetz, M. (2009). Scheduling jobs with time-resource tradeoff via nonlinear programming. *Discrete Optimization*, 6, 414–419.
- Grötschel, M., Lovász, L., & Schrijver, A. (1988). Complexity, oracles, and numerical computation. In *Geometric algorithms and combinatorial optimization*. Springer. 0-387-13624-X.
- Hall, N. G., & Posner, M. E. (2001). Generating experimental data for computational testing with machine scheduling applications. *Operations Research*, 49(6), 854–865.
- Hsieh, P.-H., Yang, J.-J., & Yang, D.-L. (2015). Decision support for unrelated parallel machine scheduling with discrete controllable processing times. *Applied Soft Computing*, 30, 475–483.
- ISO. (2013). *ISO 50001 energy management*. Available at: <https://www.iso.org/iso-50001-energy-management.html>. Accessed 22 July 2025
- Jansen, K., Mastrolilli, M., & Solis-Oba, R. (2001). Job shop scheduling problems with controllable processing times. In *Theoretical computer science. ICTCS 2001. Lecture notes in computer science* (Vol. 2202, pp. 107–122). Springer.
- Kwon, W.-C., & Kim, T. (2005). Optimal voltage allocation techniques for dynamically variable voltage processors. *ACM Transactions on Embedded Computing Systems*, 4(1), 211–230.
- Lei, M., & Feng, Z. (2012). A proposed grey model for short-term electricity price forecasting in competitive power markets. *Electrical Power and Energy Systems*, 43, 531–538.
- Li, M., & Yao, F. F. (2006). An efficient algorithm for computing optimal discrete voltage schedules. *SIAM Journal on Computing*, 35(3), 658–671.
- Liu, S. (2024). A survey of dynamic voltage and frequency scaling for high-performance low-power systems. In *2024 International conference on engineering technology and medicine research (ICETMR 2024)* (pp. 73–81).
- Mickovic, A., & Wouters, M. (2020). Energy costs information in manufacturing companies: A systematic literature review. *Journal of Cleaner Production*, 254, Article 119927.
- Miller, R. (2021). *The importance of energy efficiency for industry*. energycentral.com. Available at: <https://www.linkedin.com/pulse/importance-energy-efficiency-industry-ron-miller-pe-mba-cem-rep>. Accessed 28 November 2025
- Mitra, S., Pinto, J. M., & Grossmann, I. E. (2014). Optimal multi-scale capacity planning for power-intensive continuous processes under time-sensitive electricity prices and demand uncertainty. Part II: Enhanced hybrid bi-level decomposition. *Computers and Chemical Engineering*, 65, 102–111.
- Mochocki, B., Hu, X. S., & Quan, G. (2002). A realistic variable voltage scheduling model for real-time applications. In *IEEE/ACM international conference on computer aided design, San Jose, CA*.
- Moon, J.-Y., Shin, K., & Park, J. (2013). Optimization of production scheduling with time-dependent and machine-dependent electricity cost for industrial energy efficiency. *International Journal of Advanced Manufacturing Technology*, 68, 523–535.
- Mouzon, G., & Yildirim, M. B. (2008). A framework to minimise total energy consumption and total tardiness on a single machine. *International Journal of Sustainable Engineering*, 1, 105–116.
- Mouzon, G., Yildirim, M. B., & Twomey, J. (2007). Operational methods for minimization of energy consumption of manufacturing equipment. *International Journal of Production Research*, 45, 4247–4271.
- Orientech Automation Solutions. (2020). *8 steps to follow for proper shutdown of your manufacturing line*. Available at: <https://orientech.com/8-steps-to-follow-for-proper-shutdown-of-your-manufacturing-line/>. Accessed 28 November 2025
- Rao, R., & Vrudhula, S. (2005). Energy optimal speed control of devices with discrete speed sets. In *42nd ACM/IEEE design automation conference (DAC 2005), June 13–17, Anaheim, California*.
- Sharma, A., Zhao, F., & Sutherland, J. W. (2015). Ecological scheduling of a manufacturing enterprise operating under a time-of-use electricity tariff. *Journal of Cleaner Production*, 108, 256–270.
- Shrouf, F., Ordieres-Meré, J., García-Sánchez, A., & Ortega-Mier, M. (2014). Optimizing the production scheduling of a single machine to minimize total energy consumption costs. *Journal of Cleaner Production*, 62, 197–207.
- U.S. Department of Commerce. (2014). *The energy industry in the United States*.
- U.S. Energy Information Administration. (2025). *Using and saving energy in industry*. Available at: <https://www.eia.gov/kids/using-and-saving-energy>. Accessed 28 November 2025
- Wright, G. (2013). Reducing peak demand: Lowering prices, but what about emissions? *The Conversation*. Available at: <https://theconversation.com/reducing-peak-demand-lowering-prices-but-what-about-emissions-11564#:~:text=>. Accessed 28 November 2025
- Zhang, H., Zhao, F., Fang, K., & Sutherland, J. W. (2014). Energy-conscious flow shop scheduling under time-of-use electricity tariffs. *CIRP Annals Manufacturing Technology*, 63, 37–40.
- Zhang, R., & Chiong, R. (2016). Solving the energy-efficient job shop scheduling problem: A multi-objective genetic algorithm with enhanced local search for minimizing the total weighted tardiness and total energy consumption. *Journal of Cleaner Production*, 112, 3361–3375.
- Zhang, S.-X., Cao, Z.-G., & Zhang, Y.-Z. (2006). Scheduling with discretely compressible processing times to minimize makespan. In *The Sixth International Symposium on Operations Research and its Applications, Xinjiang, China* (pp. 258–263).
- Zhang, S. X., & Zhang, Y. Z. (2013). Single machine scheduling with discretely compressible processing times. *Advanced Materials Research*, 787, 1020–1024.